

Task 2

Tomorrow's dispatch plan

Take-home assessment

Roughly one-third of your total time budget goes here.

Our field reps visit pharmacies every day to collect orders in person. Each rep works a single area and knows it by heart; what nobody knows by heart is the *order* to run it in. You will change that for all of them, for one specific day.

What we want to see at the end

One sentence: **we open your website, pick any of our four areas, and see that employee's day planned out** — where they start, the stops in the order you chose, when they arrive at each one, and where the day ends — with timings we can trust because they come from the history, not from guesswork.

Ground rules

1. **Offline code only.** No network calls, no model APIs inside submitted code.
2. **One command reruns everything.** Deterministic, from a clean folder. We re-run your pipeline and compare against what the site shows.
3. **Honesty about gaps.** Partial and true beats complete and invented.
4. **Ask, then proceed.** Ambiguity: ask early, assume explicitly, keep moving.
5. **Trust nothing blindly** — including the data, and your own first results.
6. Using AI assistants is allowed and expected; keep an honest `AI_DISCLOSURE.md`. Fabricating results or disclosure ends the conversation.

The data

CSVs for Task 2 sit in `task2/dataset/` in this repository: the pharmacy registry, the areas we operate in, the full visit log of our field reps, invoice headers from all three order systems, and tomorrow's requested stops per area.

The situation

We have **four field employees**, each permanently assigned to one area. Tomorrow morning, **each of them must visit the 16 stops assigned to their area** — `route_plan_tomorrow.csv` names the area, the employee, and the stops. You build the plan for every employee. Fixed conditions: each rep starts and ends the day at their area's map center, lunch is 25 minutes, and everybody must be back by 17:40.

The deliverable: a working route viewer on honest plans

File structure is your call — we grade what we can *do*. Your `task2/` folder on your fork holds your analysis pipeline, your four plans, and:

A `run.bat` in the root of your `task2` folder. We double-click it, a website opens on localhost, we pick an area, and we see that employee's planned day. If it does not launch on a clean machine, nothing else gets graded. Streamlit will get you there fastest; any web stack is welcome.

When we browse it, here is what we expect:

1. **Pick an area, see the day.** Start point, end point, and the planned path as connected dots, stop by stop, in your planned order. It does not need cartographic realism — dots joined by lines are fine. It does need to reflect YOUR plan: if the plan changes, the picture changes.
2. **Timings that come from somewhere.** For every stop: when the rep arrives, how long they stay, and how long the drive really took to get there – derived from the visit log's good days and the invoices. The log is honest about some things and sloppy about others; quantify what is missing or contradictory before trusting any average, and be able to show what your correction changed.
3. **A rhythm, not a dice roll.** Pharmacies do not order at random hours. Each one has a pattern; a visit that lands before the pattern is useful, a visit after it is a wasted drive. Set per-stop timing targets from the invoices — all three systems count — and where a stop's history is thin, say so instead of inventing a number.
4. **Proof the day fits.** Arithmetic, printed by a script: every stop, every drive, lunch, and the return – inside the 17:40 cutoff. Not prose.
5. **Dispatcher judgment.** Real dispatch days include stops that cannot be served normally. If your preparation reveals any such stop — in any area — handle it the way a good dispatcher would before customers notice: substitute, reorder, or skip with cause, and write down the policy. Also tell us, per area, which single-stop failure hurts that day most.

What “good” means here. There is no single correct sequence. A plan that respects each pharmacy’s rhythm, is built on history you first made trustworthy, states its assumptions numerically, and survives its own stress test beats a prettier tour built on data you never questioned. We do not care what your intermediate files look like; we care that the plans on the site survive our re-run of your pipeline.